

# G-DMAD: Multi-Agent System Architecture

Group-Based Diverse Multi-Agent Debate for Robust Reasoning — Component Roles & Interconnections

## 1. Executive Summary

The **G-DMAD** system orchestrates specialized cognitive agents across multiple parallel groups and debate rounds. It eliminates single-agent cognitive blind spots and hallucinations through **intra-group cognitive specialization**, **inter-group adversarial dialectic**, and **multi-dimensional meta-evaluation**.

## 2. System Topology & Dual-Tier Architecture

The framework operates across two coordinated layers:

- **Intra-Group Tier (Sequential Reasoning):** Inside each group, three distinct agents (IO → CCoT → DDCoT) execute in dependency order to produce a coherent group synthesis.
- **Inter-Group Tier (Concurrent Adversarial Debate):** Group 1 (Analytical & Constructive) and Group 2 (Critical & Dialectical) run in parallel threads. Their outputs are judged by the Meta-Evaluator, and the winning Leader Answer is cross-pollinated into opposing group memory for subsequent rounds.

| Stage                 | Executor / Agent                                | Function & Interaction                                                                                                                    |
|-----------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Input              | User / API Interface                            | User enters query via Web UI; dispatches request to DebateEngine.                                                                         |
| 2. Parallel Threads   | Group 1 (Constructive)<br>Group 2 (Dialectical) | Both groups run simultaneously in ThreadPoolExecutor to generate distinct analytical viewpoints.                                          |
| 3. Intra-Group Flow   | IO → CCoT → DDCoT                               | Sequential refinement: Direct Baseline → Scene Graph JSON → Sub-question Decomposition.                                                   |
| 4. Meta-Evaluation    | Meta-Evaluator (LLM)                            | Grades both group answers objectively across Evidence Grounding, Logical Coherence, Hallucination Detection, and Depth of Thought (0-40). |
| 5. Leader Selection   | Leader Selector                                 | 6-tier deterministic tie-breaker declares the winning group answer for that round.                                                        |
| 6. Cross-Round Memory | GroupContext Memory                             | Winning Leader Answer is injected into both groups' memory for Round 2 & Round 3 critique and convergence.                                |
| 7. Final Output       | Consensus Extractor                             | Final refined consensus answer is extracted after all rounds complete.                                                                    |

## 3. Detailed Agent Specifications

### 3.1 IO Agent (Input-Output Agent)

**File:** agents/io\_agent.py | **Prompt:** prompts/io\_prompt.txt

**Role:** Direct Baseline Generation without chain-of-thought overhead.

- **Operation:** In Round 1, generates a fast initial answer according to the group's perspective. In Round > 1, incorporates the previous winning Leader Answer.
- **Input:** Question text, round number, rendered group context.
- **Output:** Concise plain-text baseline answer.
- **Connection:** Feeds directly into the CCoT Agent as primary context.

### 3.2 CCoT Agent (Canonical Chain-of-Thought)

**File:** agents/ccot\_agent.py | **Prompt:** prompts/ccot\_prompt.txt

**Role:** Structural & Relational Modeling through Scene Graph Generation.

- **Operation:** Constructs a formal JSON Scene Graph mapping entities, properties, and relationships before executing step-by-step reasoning.
- **Input:** User question, round number, group context, and IO Agent answer.
- **Output:** Scene graph JSON structure + derived deductive reasoning.
- **Connection:** Output is bundled with IO Agent output and fed to the DDCoT Agent.

### 3.3 DDCoT Agent (Dynamic Decomposition Chain-of-Thought)

**File:** agents/ddcot\_agent.py | **Prompt:** prompts/ddcot\_prompt.txt

**Role:** Divide-and-Conquer Problem Decomposition & Consensus Integration.

- **Operation:** Breaks complex multi-part queries into atomic sub-questions, answers each systematically, and reconciles answers from IO and CCoT agents.
- **Input:** Question, group context, IO Agent answer, CCoT Agent answer.
- **Output:** Decomposed sub-answers + unified group resolution.
- **Connection:** Feeds into GroupManager.\_aggregate\_group\_answer().

### 3.4 Group Manager

**File:** group/group\_manager.py | **Context:** memory/context.py

**Role:** Intra-Group Coordinator, Context Pipeline & Perspective Enforcer.

- **Group 1 Perspective:** Analytical & Constructive (direct empirical proofs, foundational physics/math, affirmative derivations).
- **Group 2 Perspective:** Critical & Dialectical (counter-arguments, edge cases, assumption scrutinizing, boundary conditions).
- **Connection:** Orchestrated directly by the DebateEngine in parallel threads.

### 3.5 Meta-Evaluator

**File:** leader/meta\_evaluator.py | **Prompt:** prompts/meta\_prompt.txt

**Role:** Objective 4-Dimensional Multi-Metric Arbiter.

- **Evaluation Dimensions (0-10 each, Total 0-40):**
  1. **Evidence Grounding (EG):** Factual accuracy and rigorous citation/formula support.
  2. **Logical Coherence (LC):** Soundness of deductive steps and absence of contradictions.
  3. **Hallucination Detection (HD):** Freedom from fabricated assumptions or calculation errors.
  4. **Depth of Thought (DT):** Thoroughness, nuance, and exploration of counter-perspectives.
- **Resilient JSON Parser:** Handles raw JSON, markdown code fences (``json), and heuristic fallbacks.
- **Connection:** Receives answers from both groups; feeds scores to LeaderSelector.

### 3.6 Leader Selector

**File:** leader/leader\_selector.py

**Role:** Deterministic Multi-Tier Decision Maker & Unbiased Tie-Breaker.

- **Selection Logic:** Primary: Total Score → Secondary: Depth of Thought → Tertiary: Evidence Grounding → Quaternary: Logical Coherence → Quinary: Reasoning Detail → Senary: Round Rotation.
- **Connection:** Declares winner\_index; signals DebateEngine to broadcast winner.

## 4. End-to-End Inter-Agent Data Flow

| Step | Sender         | Receiver              | Data Payload                           |
|------|----------------|-----------------------|----------------------------------------|
| 1    | User / UI      | DebateEngine          | Query string, rounds, settings         |
| 2    | DebateEngine   | Group 1 & 2 (Threads) | Question, round number, leader context |
| 3    | GroupManager   | IO Agent              | Question + Rendered Context            |
| 4    | IO Agent       | CCoT Agent            | IO Answer string                       |
| 5    | CCoT Agent     | DDCoT Agent           | IO Answer + CCoT Scene Graph           |
| 6    | DDCoT Agent    | GroupManager          | Decomposed resolution                  |
| 7    | GroupManager   | DebateEngine          | Aggregated Group Answer                |
| 8    | DebateEngine   | Meta-Evaluator        | [Group 1 Answer, Group 2 Answer]       |
| 9    | Meta-Evaluator | LeaderSelector        | Scores: EG, LC, HD, DT, Total          |
| 10   | LeaderSelector | DebateEngine          | Winner index (1 or 2)                  |
| 11   | DebateEngine   | GroupContext Memory   | Winning Leader Answer (for Round R+1)  |
| 12   | DebateEngine   | Web UI (SSE Stream)   | Live metrics, timer, tokens, answers   |